

Análise de Algoritmos

Inserção sem duplicidade em arquivos CSV

Busca sequencial × MergeSort com busca binária — versões iterativas e recursivas: análise de complexidade e verificação experimental

Base de dados: registros de objeto do Inmetro — *Andadores Infantis* (Portal de Dados Abertos, dados.gov.br)

Integrantes

Caroline Zaiatz - UC24101689

Beatriz Nascimento Costa Medina - UC24101412

Eloisa Sousa de Jesus - UC23100484

Professor: Zoe Roberto Magalhães Junior

Repositório: <https://github.com/BeatrizMedinna/Relatorio4>
2026

1. Objetivo

Este trabalho implementa, em linguagem C, um programa que acrescenta ao final de um arquivo CSV de destino os registros contidos em outro arquivo CSV, recusando todo registro cujo valor na coluna-chave já exista. A verificação de duplicidade é resolvida de duas maneiras: **Solução 1**, com busca sequencial (iterativa e recursiva), e **Solução 2**, carregando as chaves em um vetor na memória principal, ordenando-o com MergeSort (iterativo e recursivo) e consultando-o com busca binária (iterativa e recursiva). Para cada algoritmo são demonstradas as complexidades de tempo e de espaço, e as previsões teóricas são confrontadas com medições de tempo, contagens de comparações e profundidades de pilha obtidas experimentalmente.

Notação usada em todo o texto: n é o número de registros já existentes no destino, m o número de registros no arquivo de novos, $\lg x = \log_2 x$, e L o tamanho médio de uma chave (aqui, 11 caracteres). Como L é constante, uma comparação de chaves (strcmp) custa $O(1)$ e é a **operação dominante** contada na análise.

2. Base de dados

Foi usado o conjunto de dados do Inmetro com os **registros de objeto** do programa de avaliação da conformidade *Andadores Infantis*, publicado no Portal de Dados Abertos (<https://dados.gov.br/dados/conjuntos-dados/registro-de-objetos>). Cada linha descreve um item vinculado a um registro concedido pelo Inmetro: situação do registro, datas, dados do fornecedor (razão social, CNPJ, endereço, e-mail), fabricante, portaria aplicável, certificado, país de origem e a descrição do item (marca, modelo, descrição, códigos de barras e situação do item).

Característica	Valor
Arquivo	ANDADORES_INFANTIS.csv (guardado sem alterações em dados/original/)
Registros / colunas	93 linhas de dados + cabeçalho / 32 colunas
Separador / quebra de linha	ponto e vírgula (;) / CRLF
Codificação	UTF-16 LE com BOM (FF FE); campos vazios preenchidos com o caractere NUL (U+0000)
Período	registros concedidos de 2019 a 2026
Colunas principais	NumeroRegistro, Status, DataConcessao, RazaoSocial, CnpjCpf, Municipio, Uf, Fabricante, Certificado, PaisOrigem, ItemMarca, ItemModelo, ItemDescricao, ItemCodigoBarra, ItemStatus

Tabela 1 – Características do arquivo original.

Duas particularidades do arquivo exigiram tratamento. Primeiro, a codificação UTF-16 faz cada caractere ASCII ocupar dois bytes (um deles nulo), o que quebra qualquer leitura com funções de string de C. Por isso o repositório inclui ferramentas/converte_utf16.c, que converte o arquivo para UTF-8, troca CRLF por LF e elimina os caracteres NUL dos campos vazios, gerando dados/andadores_utf8.csv. O programa principal detecta arquivos UTF-16 e pede a conversão em vez de produzir resultados errados. Segundo, algumas descrições contêm vírgulas, mas nenhuma contém ponto e vírgula nem aspas; ainda assim, o leitor de campos respeita campos entre aspas.

3. Coluna usada como chave de unicidade

A chave escolhida é **NumeroRegistro** (coluna 0), no formato NNNNNN/AAAA (número sequencial do registro / ano de concessão). É o identificador que o Inmetro atribui a cada

registro de objeto: dois registros distintos não podem ter o mesmo número, portanto é uma coluna cujos valores *devem* ser únicos. Os valores **não estão ordenados** no arquivo: as primeiras linhas são 010691/2026, 010280/2026, 010693/2026, 010522/2026, 010524/2026, 007770/2026... — a sequência sobe e desce, e comparar a coluna com sua versão ordenada confirma que ela está fora de ordem.

Ao analisar o arquivo, verificou-se que ele tem **93 linhas mas apenas 75 números de registro distintos**: 8 registros aparecem em mais de uma linha (18 linhas excedentes), porque o Inmetro publica uma linha por *item* do registro, incluindo versões anteriores de itens já marcados como “Excluído” (Tabela 2). Ou seja, o arquivo bruto não é uma tabela de registros, e sim de itens. Para obter uma base de destino na qual a chave é de fato única — pré-requisito do problema — a base foi construída pelo próprio programa: partindo de um destino inexistente, os 93 registros foram inseridos; o programa aceitou 75 e rejeitou as 18 repetições (mantendo a primeira ocorrência de cada número). O resultado é `dados/base_andadores.csv`, que serve de destino nos testes com dados reais (comando `make base`).

NumeroRegistro	Linhas no arquivo	Motivo da repetição
001472/2022	7	6 versões excluídas + 1 item vigente
001803/2023	4	3 versões excluídas + 1 item vigente
005052/2021	4	3 versões excluídas + 1 item vigente
009477/2019	3	2 versões excluídas + 1 item vigente
001185/2024, 001636/2023, 006227/2019	2 cada	1 versão excluída + 1 item vigente
005630/2021	2	dois itens vigentes (Ref. 914 e Ref. 915)

Tabela 2 – Números de registro repetidos no arquivo original.

4. Implementação

O programa insere recebe na linha de comando o arquivo de novos registros e o arquivo de destino, nessa ordem (`./bin/insere novos.csv destino.csv`). A opção `-m` escolhe um dos seis métodos: `seq-iter`, `seq-rec` (Solução 1) e `bin-X-Y` (Solução 2), em que `X` é a versão do MergeSort e `Y` a da busca binária (`iter` ou `rec`). Há ainda `-k` (coluna-chave), `-v` (lista os rejeitados), `-s` (simulação, sem gravar) e `-t` (métricas). O fluxo é:

- **Carga:** lê o destino linha a linha e guarda apenas as chaves em um vetor de `Item {char *chave; long idx;}`; lê o arquivo de novos guardando as linhas completas (precisam ser gravadas depois) e suas chaves. As chaves são comparadas sem espaços nas pontas.
- **Verificação:** cada registro novo é classificado como aceito, duplicado no destino, duplicado dentro do próprio lote ou inválido (chave vazia/coluna ausente). A verificação dentro do lote é indispensável: sem ela, um arquivo de novos com duas linhas de mesma chave inédita introduziria uma duplicata no destino.
- **Gravação:** os aceitos são acrescentados ao final do destino (modo *append*), na ordem original, com o mesmo tipo de quebra de linha do destino; se o destino não existe, é criado com o cabeçalho.

Premissa comum às duas soluções. Nas duas soluções as chaves do destino são lidas do disco uma única vez e mantidas em RAM; o que as diferencia é o algoritmo de verificação. Uma busca sequencial que relesse o arquivo de destino para cada registro novo teria o mesmo $\Theta(m \cdot n)$ comparações, porém multiplicaria também a leitura de disco

por m , o que misturaria custo de E/S com custo de algoritmo e inviabilizaria a comparação. A leitura dos arquivos, $\Theta(n + m)$ linhas, é idêntica nos seis métodos e é medida à parte.

Instrumentação. Toda comparação de chaves incrementa um contador global, e toda chamada recursiva atualiza a profundidade atual e a máxima da pilha. Esse custo $O(1)$ é o mesmo em todas as versões. Nas funções recursivas o decremento da profundidade ocorre depois da chamada, de modo que ela não é a última instrução: o compilador não pode convertê-la em laço (*tail call*), e a recursão medida é real.

Validação. O script `testes/testes_funcionais.sh` executa os seis métodos sobre os mesmos dados (base real + `novos_exemplo.csv`, com 12 casos: inéditos, já existentes, chave com espaços, repetição no lote e chave vazia; construção da base a partir do CSV real; e um conjunto sintético com 5 000 + 2 000 linhas) e confere que os arquivos resultantes são idênticos byte a byte, que uma segunda execução não insere nada e que o destino final não tem chaves repetidas. Todos os testes passaram, e a execução com AddressSanitizer não acusou erros de memória.

5. Solução 1 — busca sequencial

```
long busca_sequencial_iterativa(const Item *v, long n, const char *chave) {
    for (long i = 0; i < n; i++)
        if (compara(v[i].chave, chave) == 0) return i;
    return -1;
}
static long seq_rec(const Item *v, long n, const char *chave, long i) {
    long r;
    ENTRA_RECURSAO();
    if (i >= n)                r = -1;                /* caso base */
    else if (compara(v[i].chave, chave) == 0) r = i;    /* caso base */
    else                        r = seq_rec(v, n, chave, i + 1);
    SAI_RECURSAO();
    return r;
}
```

5.1 Busca sequencial iterativa

Tempo. O laço executa uma comparação por iteração. Se a chave está na posição i (base 0), são feitas $i + 1$ comparações; se não está, são n . Logo: melhor caso 1 comparação, $\Theta(1)$; pior caso (ausente ou na última posição) n comparações, $\Theta(n)$. Supondo a chave presente em posição uniformemente distribuída, o caso médio de sucesso é

$$C_{\text{suc}}(n) = (1/n) \cdot \sum_{i=1..n} i = (n + 1)/2 = \Theta(n), \quad C_{\text{insuc}}(n) = n = \Theta(n).$$

Espaço. Além do vetor recebido (que pertence ao chamador), usa só as variáveis i , n e dois ponteiros: espaço extra $O(1)$.

5.2 Busca sequencial recursiva

Tempo. Seja $T(k)$ o custo quando restam $k = n - i$ elementos. Cada chamada faz trabalho constante c e, no pior caso, uma chamada com $k - 1$ elementos:

$$T(0) = c_0; \quad T(k) = T(k - 1) + c \Rightarrow T(k) = T(k - j) + j \cdot c \Rightarrow (j = k) \quad T(n) = c_0 + n \cdot c = \Theta(n).$$

O número de comparações é exatamente o mesmo da versão iterativa (melhor 1, pior n , média $(n+1)/2$).

Espaço. Cada chamada empilha um quadro de ativação que só é liberado quando a busca termina. No pior caso há $n + 1$ quadros simultâneos: $S(n) = (n + 1) \cdot s = \Theta(n)$, em que s é o tamanho do quadro. No binário gerado (GCC 13, -O2, x86-64) cada quadro ocupa **48 bytes** (cinco registradores salvos + endereço de retorno). Com a pilha padrão de 8 MiB

do Linux, o limite teórico é $8\,388\,608 / 48 \approx 174\,762$ chamadas; experimentalmente, a busca funcionou com 174 000 elementos e terminou com *segmentation fault* com 176 000. Esse é o efeito prático do espaço $\Theta(n)$: a versão recursiva não é apenas mais lenta, ela tem um limite de tamanho que a iterativa não tem (no Windows, com 1 MB de pilha, o limite cai para a ordem de 20 mil elementos; por isso o Makefile aumenta a pilha).

5.3 Custo do programa completo (Solução 1)

Para detectar repetições dentro do lote, cada chave aceita é acrescentada ao fim do vetor, que cresce de n até $n + A$ (A = aceitos). A i -ésima busca examina no máximo $n + i$ chaves:

$$C_1(n, m) \leq \sum_{i=0..m-1} (n + i) = m \cdot n + m(m - 1)/2 = O(m \cdot n + m^2) (= O(m \cdot n) \text{ quando } m \leq n).$$

Se uma fração p dos novos já existe no destino (custo médio $(n+1)/2$) e o restante precisa percorrer o vetor inteiro, o valor esperado é aproximadamente $D(n+1)/2 + (A+L)(n + A/2)$, com D , L e A = quantidades de duplicados no destino, repetidos no lote e aceitos. Espaço total: vetor de $n + m$ ponteiros + as chaves = $\Theta(n + m)$; a versão recursiva acrescenta até $n + m$ quadros de pilha, também $\Theta(n + m)$.

6. Solução 2 — MergeSort e busca binária

As n chaves do destino são carregadas em um vetor em RAM e ordenadas. As chaves válidas do lote também são ordenadas com o mesmo MergeSort: como ele é **estável**, chaves iguais ficam vizinhas e a primeira ocorrência (menor posição original) vem antes, de modo que as repetições internas do lote são detectadas por uma única varredura de vizinhos ($m - 1$ comparações). Por fim, cada chave do lote é procurada no vetor do destino por busca binária.

```
static void intercala(Item *v, Item *aux, long ini, long meio, long fim) { /*  $\Theta(\text{fim}-\text{ini}+1)$  */
    long i = ini, j = meio + 1, k = ini;
    while (i <= meio && j <= fim)
        aux[k++] = (compara(v[i].chave, v[j].chave) <= 0) ? v[i++] : v[j++]; /* <= : estável */
    while (i <= meio) aux[k++] = v[i++];
    while (j <= fim) aux[k++] = v[j++];
    memcpy(v + ini, aux + ini, (fim - ini + 1) * sizeof(Item));
}

static void ms_rec(Item *v, Item *aux, long ini, long fim) { /* top-down */
    if (ini < fim) { long meio = ini + (fim - ini) / 2;
        ms_rec(v, aux, ini, meio); ms_rec(v, aux, meio + 1, fim); intercala(v, aux, ini, meio, fim); }
}

int mergesort_iterativo(Item *v, long n) { /* bottom-up */
    Item *aux = malloc(n * sizeof(Item));
    for (long larg = 1; larg < n; larg *= 2)
        for (long ini = 0; ini < n - larg; ini += 2 * larg) {
            long fim = min(ini + 2 * larg - 1, n - 1);
            intercala(v, aux, ini, ini + larg - 1, fim);
        }
    free(aux); return 0;
}
```

6.1 MergeSort recursivo (top-down)

Tempo. A intercalação de dois blocos que somam k elementos faz no máximo $k - 1$ comparações e $2k$ movimentos: $\Theta(k)$. Dividir custa $O(1)$. Assim,

$$T(1) = c; T(n) = T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + \Theta(n).$$

Pela árvore de recursão: o nível j tem 2^j subproblemas de tamanho $n/2^j$, somando $c \cdot n$ por nível; há $\lceil \lg n \rceil$ níveis de intercalação, logo $T(n) = c \cdot n \lceil \lg n \rceil + \Theta(n) = \Theta(n \log n)$. O mesmo resultado sai do Teorema Mestre ($a = 2$, $b = 2$, $f(n) = \Theta(n) = \Theta(n^{\log_b a})$, caso 2). O custo não depende da ordem da entrada: melhor, médio e pior caso são $\Theta(n \log n)$. Em

comparações, o pior caso é exatamente $n\lceil \lg n \rceil - 2^{\lceil \lg n \rceil} + 1$ e o caso médio vale aproximadamente $n \lg n - 1,26n$ (Knuth).

Espaço. O vetor auxiliar tem n posições (16 bytes cada: ponteiro + índice), $\Theta(n)$. A pilha tem profundidade máxima $\lceil \lg n \rceil + 1$, $\Theta(\log n)$. Total: $\Theta(n) + \Theta(\log n) = \Theta(n)$.

6.2 MergeSort iterativo (bottom-up)

Tempo. O laço externo percorre larguras 1, 2, 4, ..., enquanto largura $< n$: são $\lceil \lg n \rceil$ passadas. Em cada passada, os blocos intercalados são disjuntos e cobrem o vetor, então a passada custa $\Theta(n)$. Total: $\lceil \lg n \rceil \cdot \Theta(n) = \Theta(n \log n)$, também independente da entrada. A diferença em relação à versão recursiva está na forma da árvore de intercalações: quando n não é potência de 2, o bottom-up intercala blocos de tamanhos mais desiguais no final (por exemplo, 32 768 com 17 232 para $n = 50\,000$), o que muda ligeiramente o número de comparações, mas não a ordem de grandeza.

Espaço. Mesmo vetor auxiliar $\Theta(n)$ e apenas variáveis de laço, $O(1)$: total $\Theta(n)$, **sem pilha**. As duas versões têm portanto a mesma complexidade assintótica de tempo e de espaço; a recursiva paga $\Theta(\log n)$ de pilha (21 quadros para $n = 10^6$, desprezível) e o custo das chamadas de função.

6.3 Busca binária iterativa

```
long busca_binaria_iterativa(const Item *v, long n, const char *chave) {
    long ini = 0, fim = n - 1;
    while (ini <= fim) {
        long meio = ini + (fim - ini) / 2;          /* evita overflow de ini+fim */
        int c = compara(v[meio].chave, chave);
        if (c == 0) return meio;
        if (c < 0) ini = meio + 1; else fim = meio - 1;
    }
    return -1;
}
```

Tempo. Se o intervalo tem k elementos, após uma iteração sem sucesso restam no máximo $\lfloor k/2 \rfloor$. Depois de t iterações restam no máximo $\lfloor n/2^t \rfloor$ elementos, e o laço termina quando o intervalo fica vazio; logo $t \leq \lceil \lg n \rceil + 1$. Melhor caso: 1 comparação, $\Theta(1)$; pior caso e busca sem sucesso: $\lceil \lg n \rceil + 1$ comparações, $\Theta(\log n)$; média de sucesso $\approx \lg n - 1$.

Espaço. Apenas *ini*, *fim*, *meio* e *c*: $O(1)$.

6.4 Busca binária recursiva

Tempo. Cada chamada faz trabalho constante e, se não encontra, chama a si mesma sobre metade do intervalo:

$$T(0) = c; T(n) = T(\lfloor n/2 \rfloor) + c \Rightarrow T(n) = c(\lceil \lg n \rceil + 2) = \Theta(\log n)$$

(Teorema Mestre com $a = 1$, $b = 2$, $f(n) = \Theta(1) = \Theta(n^0)$: caso 2). O número de comparações é idêntico ao da versão iterativa.

Espaço. Até $\lceil \lg n \rceil + 2$ quadros simultâneos na pilha (o último é o caso base com intervalo vazio): $\Theta(\log n)$, contra $O(1)$ da iterativa.

6.5 Custo do programa completo (Solução 2)

$$T_2(n, m) = \Theta(n \log n) [\text{ordena destino}] + \Theta(m \log m) [\text{ordena lote}] + \Theta(m) [\text{vizinhos}] + m \cdot \Theta(\log n) [\text{buscas}] = \Theta(n \log n + m \log m + m \log n)$$

Espaço: vetores de n e m itens mais o auxiliar do MergeSort (liberado ao fim de cada ordenação), $\Theta(n + m)$, mais $O(\log n)$ de pilha nas versões recursivas.

7. Comparação teórica entre as soluções

Algoritmo / programa	Tempo (melhor)	Tempo (pior)	Espaço extra
Busca sequencial iterativa	$\Theta(1)$	$\Theta(n)$	$O(1)$
Busca sequencial recursiva	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$ - pilha
MergeSort recursivo	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n)$ aux + $\Theta(\log n)$ pilha
MergeSort iterativo	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n)$ aux
Busca binária iterativa	$\Theta(1)$	$\Theta(\log n)$	$O(1)$
Busca binária recursiva	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$ - pilha
Solução 1 (m inserções)	$\Theta(m)$	$\Theta(m \cdot n + m^2)$	$\Theta(n+m)$; rec.: + $\Theta(n+m)$ pilha
Solução 2 (m inserções)	$\Theta(n \log n + m \log m)$	$\Theta(n \log n + m \log m + m \log n)$	$\Theta(n+m)$; rec.: + $\Theta(\log n)$ pilha

Tabela 3 - Resumo das complexidades (n = destino, m = novos).

Para m proporcional a n , a Solução 1 é quadrática, $\Theta(n^2)$, e a Solução 2 é $\Theta(n \log n)$: a vantagem da Solução 2 cresce sem limite. Mas a Solução 2 paga o custo fixo de ordenar o destino mesmo quando há poucos registros novos. Igualando os custos esperados em comparações (com metade dos novos já presentes, a Solução 1 gasta cerca de $0,75 \cdot n$ por registro) obtém-se o ponto de equilíbrio

$$0,75 \cdot m \cdot n \approx n \lg n - 1,26n \Rightarrow m^* \approx (\lg n - 1,26) / 0,75 \approx 19 \text{ para } n = 50\,000.$$

Abaixo de algumas dezenas de registros novos, a busca sequencial deve ser mais barata; acima disso, a ordenação se paga. Quanto ao espaço, as duas soluções são $\Theta(n + m)$; a diferença está nas versões recursivas: a busca sequencial recursiva usa pilha linear, enquanto as recursões da Solução 2 são logarítmicas.

8. Resultados experimentais

8.1 Metodologia

Ambiente: Intel Xeon 2,1 GHz (1 núcleo), 3,9 GB de RAM, Ubuntu 24.04, GCC 13.3 com `-std=c11 -O2`. Os tempos foram medidos com relógio monotônico (`clock_gettime`) dentro do programa, separando leitura dos arquivos, ordenação, buscas e gravação; cada ponto é a **mediana de 5 execuções**, com a opção `-s` (sem gravar) para que todas as execuções usem o mesmo destino. Também foram registrados o número de comparações e a profundidade máxima de recursão, que não dependem da máquina.

Como a base real tem só 75 registros (tempos de microssegundos, dominados por ruído), os experimentos usaram arquivos gerados por `ferramentas/gera_dados.c`: cada linha é a cópia de uma linha **real** da base, sorteada, com o `NumeroRegistro` trocado por uma chave única no mesmo formato `NNNNNN/AAAA`, em ordem pseudoaleatória. Nos arquivos de novos, 50% das chaves já existem no destino, 5% repetem uma chave anterior do próprio lote e o restante é inédito. Com semente fixa, os arquivos são reproduzíveis e estão no repositório (`dados/experimentos/`, até 34 MB).

- **Exp. 1** - programa completo, $m = 1\,000$ fixo, $n \in \{1\,000, 2\,000, 5\,000, 10\,000, 20\,000, 50\,000\}$.
- **Exp. 2** - programa completo, $n = 50\,000$ fixo, m de 1 a 5 000.
- **Exp. 3** - algoritmos isolados em memória (`ferramentas/bench_algoritmos.c`, mesmas funções de `src/`), n até 10^6 ; 1 000 consultas por tamanho, metade presentes.

8.2 Experimento 1 — tempo em função de n

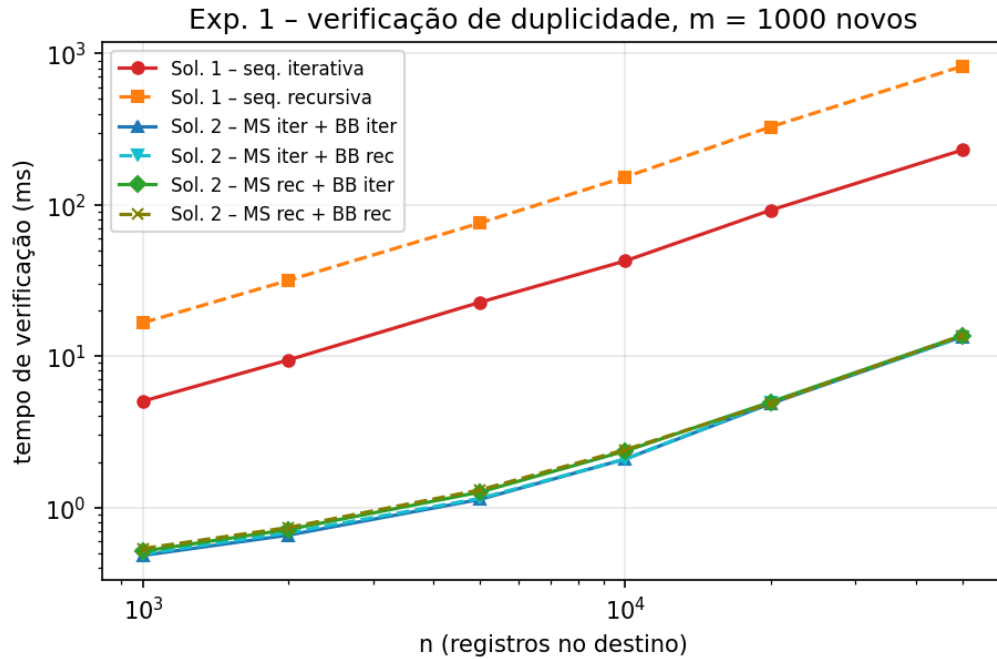


Figura 1 - Tempo de verificação (ordenação + buscas) com m = 1000, escala log-log.

Método	1.000	2.000	5.000	10.000	20.000	50.000	inclinação
seq-iter	5,04	9,42	22,70	42,56	91,99	230,07	0,98
seq-rec	16,63	31,65	75,85	152,55	326,80	822,89	1,00
bin-iter-iter	0,48	0,66	1,13	2,09	4,85	13,44	0,85
bin-iter-rec	0,49	0,68	1,15	2,09	4,93	13,38	0,84
bin-rec-iter	0,51	0,71	1,26	2,36	4,96	13,69	0,84
bin-rec-rec	0,53	0,74	1,31	2,41	4,88	13,70	0,83

Tabela 4 - Tempo de verificação (ms), m = 1000. Inclinação = expoente empírico k em $t \propto n^k$ entre n = 1000 e 50 000.

Com m fixo, a teoria prevê para a Solução 1 tempo linear em n ($\Theta(m \cdot n)$), e a inclinação medida foi 0,98 (iterativa) e 1,00 (recursiva) — praticamente 1. Na Solução 2 a inclinação do tempo total ficou abaixo de 1 (0,83 a 0,85) porque, com n pequeno, pesam as parcelas que não dependem de n (ordenar o lote de 1 000 chaves e as buscas). Isolando apenas a ordenação, que é o termo dominante $\Theta(n \log n)$, a inclinação entre n = 5 000 e 50 000 foi 1,11 a 1,17, próxima de 1 e compatível com $n \log n$ (a previsão teórica nessa faixa é 1,10). Em n = 50.000, a Solução 2 levou 13,38-13,70 ms, contra 230,07 ms da busca sequencial iterativa (**16,8× a 17,2× mais rápida**) e 822,89 ms da recursiva.

n	Duplic. destino (D)	Repet. lote (L)	Aceitos (A)	Comparações medidas	Modelo teórico	Diferença
1.000	529	20	451	851.390	841.975	1,12%
2.000	518	25	457	1.597.121	1.592.396	0,30%
5.000	514	36	450	3.808.964	3.824.607	-0,41%
10.000	533	21	446	7.465.976	7.439.408	0,36%
20.000	532	23	445	14.742.212	14.784.396	-0,29%
50.000	533	18	449	36.327.664	36.780.108	-1,23%

Tabela 5 - Solução 1: comparações medidas × modelo $D(n+1)/2 + (A+L)(n + A/2)$ da seção 5.3.

A contagem de comparações da Solução 1 coincide com o modelo teórico com diferença de no máximo 1,2% (Tabela 5); o resíduo vem de as posições das chaves duplicadas serem sorteadas, e não exatamente centradas em $n/2$.

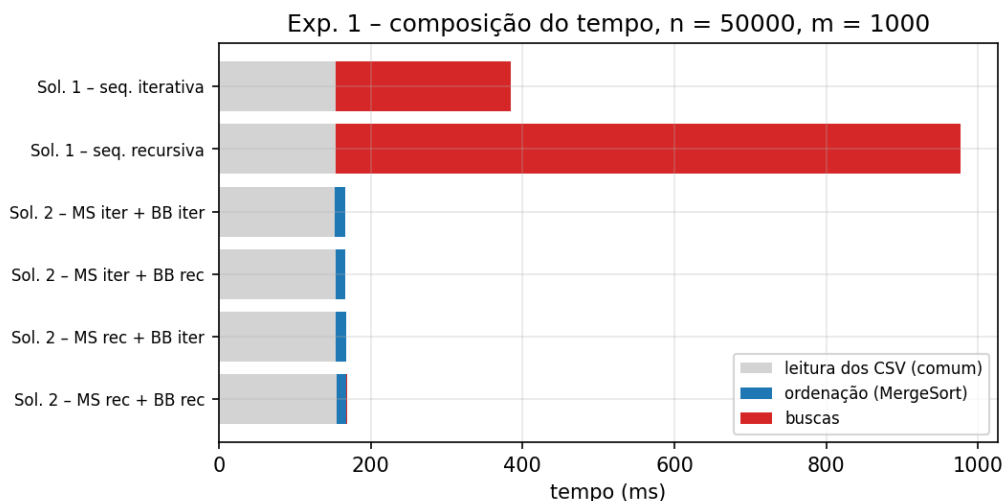


Figura 2 - Composição do tempo total para $n = 50.000$ e $m = 1000$.

A Figura 2 mostra um fato prático importante: na Solução 2 a leitura dos arquivos ($\approx 152,98$ ms para 50.000 linhas, ≈ 34 MB) custa cerca de 11 vezes mais do que ordenar e buscar. A leitura é $\Theta(n \cdot \text{tamanho da linha})$ e é inevitável; depois da ordenação, o gargalo deixa de ser o algoritmo. Na Solução 1, ao contrário, a verificação já supera a leitura com $m = 1\,000$.

8.3 Experimento 2 — ponto de equilíbrio em função de m

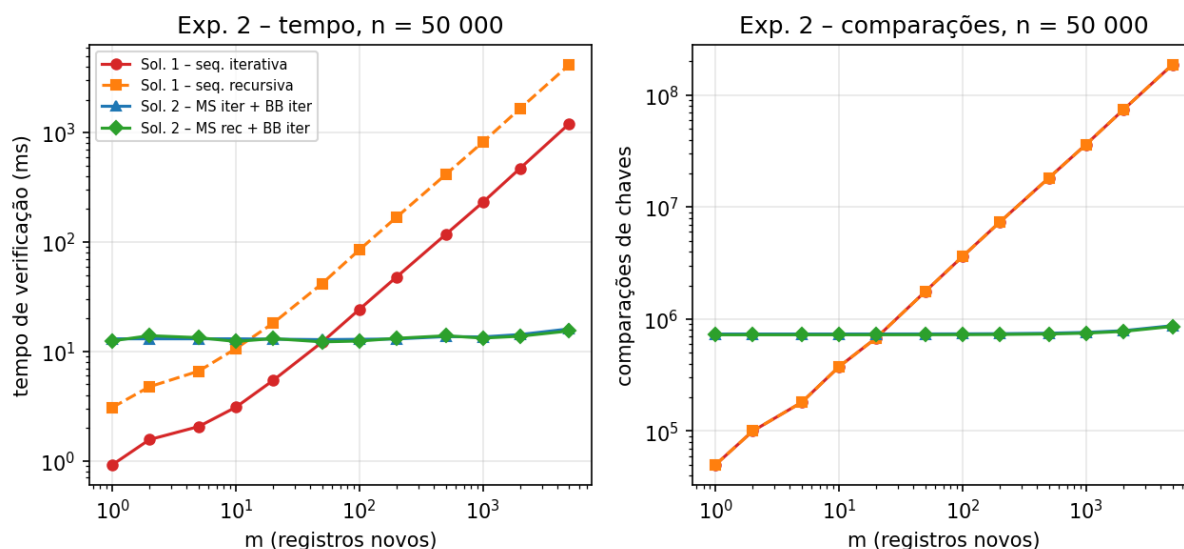


Figura 3 - $n = 50\,000$ fixo. Esquerda: tempo de verificação; direita: total de comparações (ordenação + buscas).

m	1	5	10	20	50	100	1.000	5.000
seq-iter	0,92	2,06	3,09	5,43	12,18	24,25	230,92	1.206,09
seq-rec	3,08	6,60	10,61	18,10	41,79	84,87	819,68	4.176,98
bin-iter-iter	13,04	13,04	13,04	12,95	12,82	12,91	13,58	16,07
bin-rec-iter	12,49	13,37	12,34	13,11	12,23	12,45	13,25	15,40
comp. seq	50.000	180.984	376.033	679.041	1.768.434	3.626.806	36.327.664	187.139.091
comp. bin	736.494	736.567	736.667	736.872	737.515	738.653	761.364	874.317

Tabela 6 – Exp. 2: tempo de verificação (ms) e comparações, $n = 50\,000$.

A curva da Solução 1 cresce linearmente com m , enquanto a da Solução 2 é praticamente plana: o custo está quase todo na ordenação do destino (≈ 736.478 comparações), e cada registro novo acrescenta só $\approx \lg 50\,000 \approx 16$ comparações. Em comparações, as curvas se cruzam entre $m = 20$ e $m = 50$ (interpolando, $m \approx 22$) — em acordo com a estimativa teórica $m^* \approx 19$ da seção 7. Em tempo, o cruzamento ocorre mais tarde, entre $m = 50$ e 100 (≈ 50): a comparação na busca sequencial é mais barata que a do MergeSort, porque percorre a memória em ordem (bom uso de cache e de pré-busca do processador), enquanto a intercalação também move $2n$ itens por nível. A análise assintótica prevê corretamente o formato das curvas e a ordem de grandeza do cruzamento; a posição exata depende das constantes escondidas no Θ .

8.4 Experimento 3 — MergeSort iterativo × recursivo

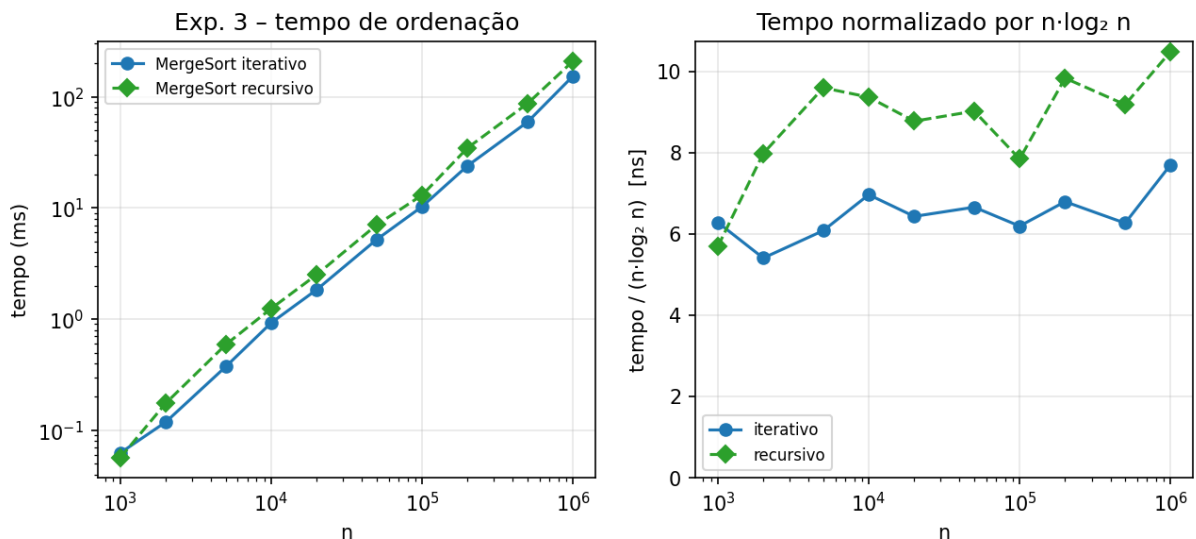


Figura 4 – Esquerda: tempo de ordenação. Direita: tempo dividido por $n \cdot \lg n$ (constante $\Rightarrow$ comportamento $\Theta(n \log n)$).

n	Comp. iterativo	Comp. recursivo	Pior caso top-down	Média aprox.	Tempo it. (ms)	Tempo rec. (ms)	rec/it
1.000	8.708	8.704	8.977	8.701	0,06	0,06	0,91
2.000	19.648	19.582	19.953	19.403	0,12	0,17	1,48
5.000	56.716	54.813	56.809	55.116	0,37	0,59	1,58
10.000	121.996	118.119	123.617	120.232	0,93	1,24	1,34
20.000	263.918	255.078	267.233	260.464	1,84	2,51	1,36
50.000	743.618	727.604	734.465	717.257	5,19	7,04	1,36
100.000	1.573.184	1.541.041	1.568.929	1.534.514	10,28	13,03	1,27
200.000	3.297.788	3.261.550	3.337.857	3.269.028	23,92	34,63	1,45
500.000	8.952.395	8.935.736	8.975.713	8.833.534	59,29	86,91	1,47
1.000.000	18.728.677	18.683.720	18.951.425	18.667.069	153,44	208,98	1,36

Tabela 7 – MergeSort: comparações medidas $\times$ fórmulas teóricas, e tempos (Exp. 3).

As comparações do MergeSort recursivo ficaram sempre abaixo do pior caso exato $n \lceil \lg n \rceil - 2^{\lceil \lg n \rceil} + 1$ e próximas da aproximação de caso médio $n \lg n - 1,26n$ (diferença máxima de 2,1%). O bottom-up faz um pouco mais de comparações quando n não é potência de 2 (em alguns tamanhos chega a ultrapassar a fórmula do top-down, que não se aplica a ele), pelas intercalações desbalanceadas descritas na seção 6.2. O tempo dividido por $n \lg n$ fica aproximadamente constante (Figura 4, direita) — a leve subida para n grande reflete o vetor deixar de caber na cache, e não uma mudança de ordem de complexidade. A versão recursiva foi em média 1,36 $\times$ mais lenta que a iterativa: mesma complexidade, mas com o custo de $\sim 2n$ chamadas de função.

8.5 Experimento 3 — buscas e espaço de pilha

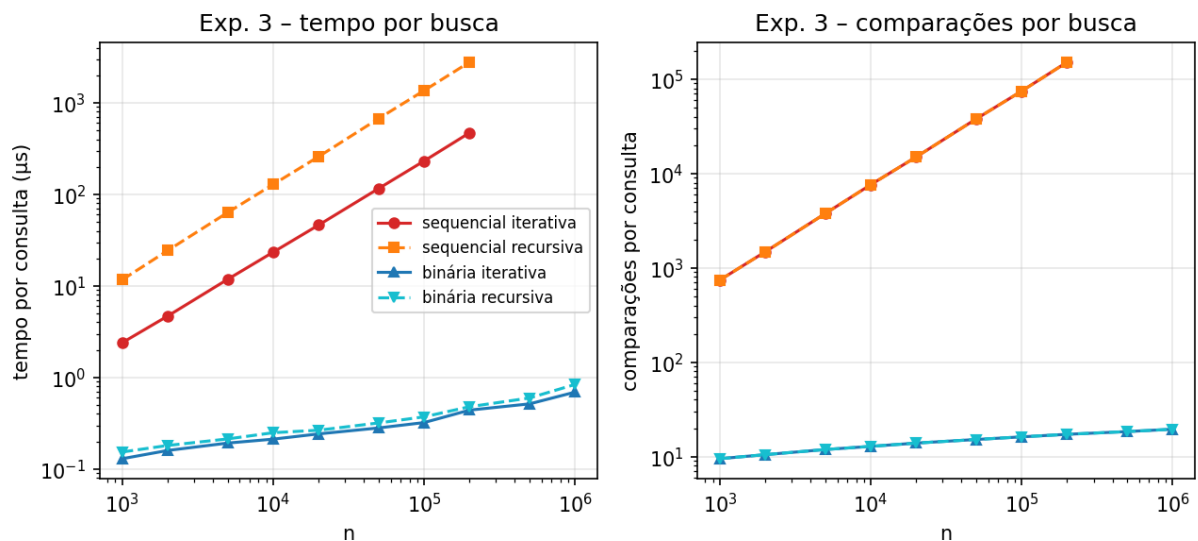


Figura 5 – Tempo e comparações por consulta (1000 consultas, metade presentes).

n	seq. it. (μs)	seq. rec. (μs)	rec/it	comp. seq (med./teor.)	bin. it. (μs)	bin. rec. (μs)	comp. bin (med./teor.)
1.000	2,4	11,7	4,9	743 / 750	0,130	0,154	9,49 / 9,48
2.000	4,7	24,4	5,2	1.480 / 1.500	0,160	0,181	10,48 / 10,48
5.000	11,8	63,5	5,4	3.766 / 3.750	0,192	0,213	11,87 / 12,14
10.000	23,4	128,4	5,5	7.611 / 7.500	0,212	0,249	12,82 / 13,14
20.000	46,2	258,4	5,6	14.975 / 15.000	0,242	0,266	13,89 / 14,14
50.000	116,0	673,5	5,8	37.928 / 37.500	0,281	0,319	15,16 / 15,30
100.000	231,5	1.361,1	5,9	74.731 / 75.000	0,322	0,372	16,17 / 16,30
200.000	470,3	2.781,6	5,9	151.509 / 150.000	0,440	0,479	17,21 / 17,30
500.000	-	-	-	-	0,515	0,597	18,40 / 18,47
1.000.000	-	-	-	-	0,692	0,839	19,48 / 19,47

Tabela 8 – Custo por consulta. Teórico sequencial: $\frac{1}{2}(n+1)/2 + \frac{1}{2}n$; binária: $\frac{1}{2}(\lceil \lg n \rceil + 1) + \frac{1}{2}(\lg n - 1)$. Sequenciais medidas até $n = 200\,000$.

As comparações por consulta confirmam a teoria: a busca sequencial cresce linearmente ($\approx 0,75n$), a binária cresce uma unidade cada vez que n dobra (de 9,5 em $n = 1\,000$ para 19,5 em $n = 10^6$, exatamente como $\lg n$). Em $n = 200\,000$, uma busca binária custa 0,44 μs contra 470 μs da sequencial — cerca de 1.069 vezes menos. A recursão custou caro na busca sequencial ($4,9\times$ a $5,9\times$ mais lenta que a iterativa), porque há uma chamada de função — com empilhamento de registradores — para cada elemento examinado; na binária a diferença foi pequena ($1,09\times$ a $1,21\times$), pois são só ~ 20 chamadas.

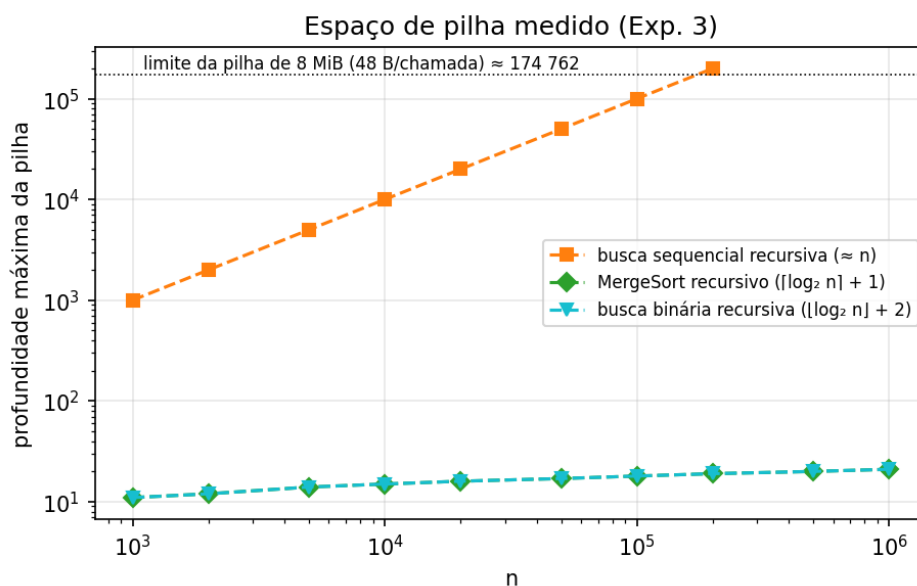


Figura 6 – Profundidade máxima de pilha medida $\times$ previsão teórica.

A profundidade de pilha medida coincidiu exatamente com a teoria em todos os tamanhos: $n + 1$ na busca sequencial recursiva, $\lceil \lg n \rceil + 1$ no MergeSort recursivo e $\lceil \lg n \rceil + 2$ na busca binária recursiva (por exemplo, 200 001, 19 e 19 para $n = 200\,000$; nos experimentos 1 e 2, a busca sequencial recursiva chegou a $n + A + 1$, incluindo as chaves aceitas do lote). A busca sequencial recursiva com $n = 200\,000$ só pôde ser executada com a pilha ampliada (`ulimit -s unlimited`); com a pilha padrão ela falha, como previsto na seção 5.2. Essa é a confirmação experimental de que o espaço $\Theta(n)$ da recursão linear é um problema real, enquanto o $\Theta(\log n)$ das recursões da Solução 2 é inofensivo.

9. Conclusões

- **Correção.** Os seis métodos produzem arquivos idênticos e nunca introduzem chaves repetidas no destino, inclusive quando a repetição está dentro do próprio arquivo de novos — caso que precisa ser tratado explicitamente.
- **Teoria × prática.** As contagens de comparações ficaram a cerca de 1% dos modelos teóricos, as profundidades de pilha coincidiram exatamente com as fórmulas, e as inclinações log-log dos tempos confirmaram o crescimento linear da Solução 1 (com m fixo) e $\Theta(n \log n)$ da ordenação.
- **Solução 1 × Solução 2.** A Solução 2 é assintoticamente superior — $\Theta((n + m) \log(n + m))$ contra $\Theta(m \cdot n)$ — e foi cerca de 17× mais rápida com $n = 50\,000$ e $m = 1\,000$. Porém ela tem um custo fixo de ordenação: para poucos registros novos (menos de ~50 com $n = 50\,000$), a busca sequencial é mais rápida. A notação assintótica indica qual solução escala; as constantes decidem qual vence em entradas pequenas.
- **Iterativo × recursivo.** Em todos os pares a complexidade de tempo é a mesma e a versão iterativa foi mais rápida. No espaço, a diferença importa quando a recursão é linear: a busca sequencial recursiva é limitada pelo tamanho da pilha (≈ 175 mil elementos em 8 MiB), enquanto MergeSort e busca binária recursivos usam poucas dezenas de quadros mesmo com um milhão de chaves.
- **Gargalo real.** Com a Solução 2, o tempo total passa a ser dominado pela leitura dos arquivos. Se as inserções forem frequentes, vale manter o índice de chaves já ordenado entre execuções (evitando reordenar o destino a cada vez) ou usar uma estrutura como tabela hash ou árvore balanceada.
- **Qualidade da base.** A coluna candidata natural a chave (NumeroRegistro) estava repetida no arquivo publicado, porque cada linha representa um item, e não um registro. Verificar essa premissa antes de implementar foi essencial para definir corretamente o que significa “não duplicar”.

10. Repositório e instruções

Código-fonte, arquivos de teste, resultados brutos e scripts:
<https://github.com/BeatrizMedinna/Relatorio4>. Compilação e execução:

```
make                                # compila bin/inserere e as ferramentas (GCC, C11)
./bin/inserere <novos.csv> <destino.csv> [-m metodo] [-k coluna] [-v] [-s] [-t]
# metodo: seq-iter | seq-rec | bin-iter-iter | bin-iter-rec | bin-rec-iter | bin-rec-rec
make base                           # converte o CSV do Inmetro e recria dados/base_andadores.csv
make testes                         # testes funcionais
make experimentos                   # experimentos 1, 2 e 3 -> resultados/
python3 relatorio/analise.py        # resumos e gráficos
python3 relatorio/gera_relatorio.py # este relatório
```

Referências

CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. *Algoritmos: teoria e prática*. 3. ed. Rio de Janeiro: Elsevier, 2012.

KNUTH, D. E. *The Art of Computer Programming, vol. 3: Sorting and Searching*. 2. ed. Addison-Wesley, 1998.

ZIVIANI, N. *Projeto de algoritmos com implementações em Pascal e C*. 3. ed. São Paulo: Cengage Learning, 2011.

BRASIL. Portal de Dados Abertos. *Registro de Objetos* (Inmetro). Disponível em: <https://dados.gov.br/dados/conjuntos-dados/registro-de-objetos>. Arquivo: ANDADORES_INFANTIS.csv.